Drone Simulation

Project Report

Autonomous Navigation and Obstacle Avoidance Simulation

Simulation Engine: MuJoCo (Multi-Joint dynamics with Contact) | **Grid Resolution:** 160 × 160 | **Physics Rate:** 200 Hz (dt = 0.005 s)

Table of Contents

1. Introduction

This project presents a comprehensive physics-driven quadcopter simulation designed to demonstrate key robotics research concepts: autonomous terrain exploration, incremental occupancy map construction, dynamic path replanning, and behaviour-based control architectures. The simulation is built on the MuJoCo physics engine, which provides high-fidelity rigid-body dynamics, contact forces, and real-time visualisation.

The environment simulates a Martian/Lunar terrain with rough heightfield topography, scattered rock obstacles, and six pre-configured landing pads. The drone navigates this environment using a combination of PID force controllers, grid-based pathfinding algorithms, and a layered behaviour management system.

Research Objectives:

1. Demonstrate autonomous terrain exploration using simulated onboard sensing with configurable sensor models (directional vs. omnidirectional).

2. Implement incremental map building with an optimistic planning strategy for unknown terrain.

3. Enable dynamic obstacle handling with real-time path invalidation and replanning.

4. Design a modular subsumption architecture that cleanly separates behaviour priorities.

5. Compare pathfinding algorithms (A* vs Dijkstra) with quantitative benchmarks.

6. Simulate realistic sensor noise and evaluate multiple state estimation filters.

2. Architecture Overview

The system follows a layered architecture with clear separation of concerns:

Layer	Module	Responsibility
Physics Engine	MuJoCo + scene.xml	Rigid-body dynamics, contact, heightfield terrain, visual rendering.
Force Controller	DronePhysicsController	PID-based altitude, horizontal position, attitude, and yaw control.
State Machine	DroneAutopilot	Flight state management (LANDED → TAKEOFF → HOVER → GOTO → AUTOLAND).
Behaviour Manager	subsumption.py	Priority-based behaviour selection (Kill Switch > Avoidance > Nav > Scan).
Terrain Scanner	terrain_scanner.py	Fog-of-war occupancy mapping with directional/omni sensor simulation.
Obstacle Manager	dynamic_obstacles.py	Runtime obstacle spawning, grid update, and FIFO recycling.
Path Planner	run_astar / run_dijkstra	8-connected grid search with obstacle inflation cost.
Sensor Suite	sensor_simulation.py	Virtual GPS, IMU, barometer, altimeter, front rangefinder with noise.
Filter Suite	KalmanFilter1D, EMA, Comp.	Kalman, EMA, and complementary state estimation filters.

3. Drone Controller

The drone is controlled exclusively through forces and torques applied to its rigid body via MuJoCo's `xfrc_applied` mechanism. No kinematic position overrides are used, making the simulation fully physics-consistent.

3.1 Altitude PID Controller

A proportional–integral–derivative (PID) controller maintains the drone's altitude. The vertical force is computed as: $\mathbf{F_z = K_p \cdot e_z + K_i \cdot \int e_z dt + K_d \cdot (de_z/dt) + F_{hover}}$, where $F_{hover} = m \cdot g$ is the hover feedforward term. The integral term is clamped to ±8.0 N to prevent windup. Controller gains: K_p=28.0, K_i=4.5, K_d=12.0.

3.2 Horizontal PD Controller

Horizontal forces use a PD controller in either position mode (waypoint tracking) or velocity mode (path following): $\mathbf{F_{xy} = K_p \cdot e_{xy} - K_d \cdot v_{xy}}$. Gains: K_p=18.0, K_d=9.0. Lateral force is clamped to ±30.0 N.

3.3 Attitude & Drag Compensation

Roll and pitch are stabilised via: $\tau = -K_{att} \cdot \theta - (K_{d,att} + K_{drag}) \cdot \omega$. Linear drag (K=1.8 N·s/m) and rotational drag (K=0.6 N·m·s/rad) are applied to simulate aerodynamic damping.

Parameter	Value	Units
Altitude K_p	28.0	—
Altitude K_i	4.5	—
Altitude K_d	12.0	—
Horizontal K_p	18.0	—
Horizontal K_d	9.0	—
Max Lateral Force	30.0	N
Integral Clamp	±8.0	N
Linear Drag	1.8	N·s/m
Rotational Drag	0.6	N·m·s/rad
Hover Height	1.5	m
Cruise Speed	1.4	m/s

4. Map Generation

The environment is discretised into a **160×160** occupancy grid covering a 16 m × 16 m arena (resolution: 0.100 m/cell). Each cell is classified as free (0) or occupied (1) based on two criteria:

1. Rock obstacles: All bodies with names starting with 'rock_' or 'dyn_' in the MuJoCo model are projected onto the grid as circular occupied regions based on their geom size.

2. Terrain height: Cells where the scaled terrain height exceeds 0.22 m are marked as impassable (representing steep terrain).

Landing pad regions are always excluded from obstacle marking to ensure navigability.

Metric	Value
Grid Dimensions	160 × 160
Total Cells	25,600
Obstacle Cells	8,354
Free Cells	17,246
Obstacle Density	32.6%
Resolution	0.100 m/cell

Figure 1: Occupancy Grid Map (with Fog-of-War overlay)

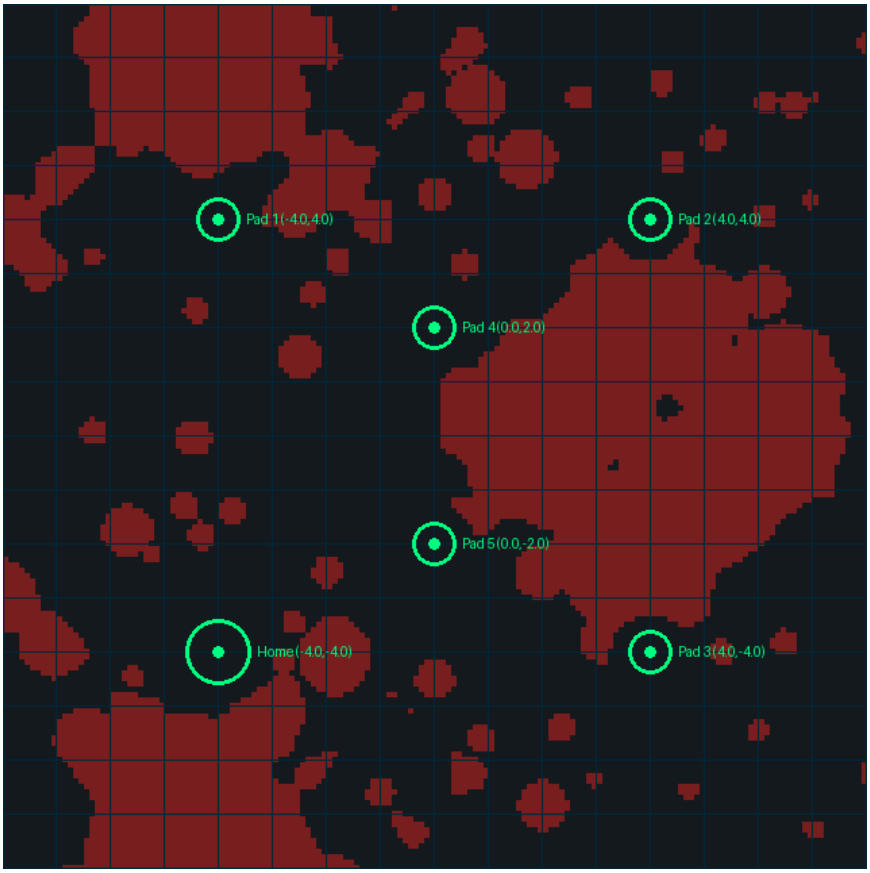
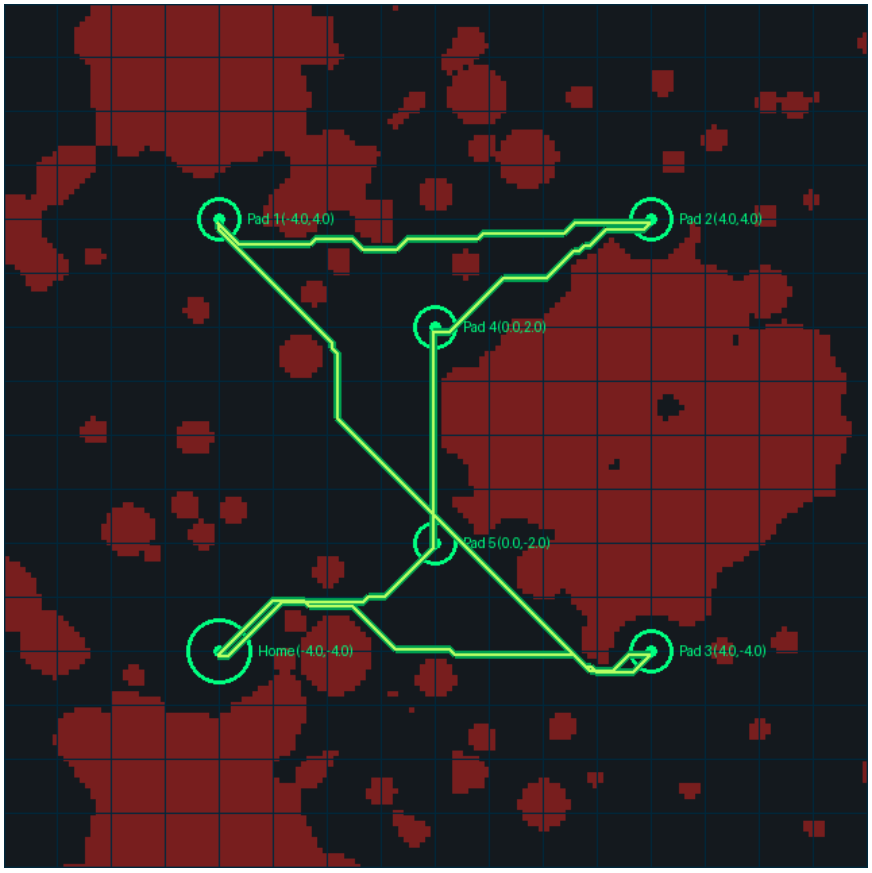


Figure 2: Planned Path Overlay on Occupancy Grid


Pad 1(-4.0,4.0)
Pad 2(4.0,4.0)
Pad 4(0.0,2.0)
Pad 5(0.0,-2.0)
Home(-4.0,-4.0)
Pad 3(4.0,-4.0)

5. Pathfinding (A* vs Dijkstra)

Both algorithms operate on the same 8-connected occupancy grid with identical movement costs. An obstacle inflation cost of +3.0 is added for cells adjacent to obstacles to maintain safe clearance during navigation.

5.1 Heuristic Design

A* uses the **Octile distance** heuristic: $h(n) = \max(|\Delta x|, |\Delta y|) + (\sqrt{2} - 1) \cdot \min(|\Delta x|, |\Delta y|)$. This is the tightest admissible heuristic for 8-connected grids with unit and diagonal ($\sqrt{2}$) costs, providing maximum pruning while guaranteeing optimality.

5.2 Canonical Benchmark (Home → Pad 2)

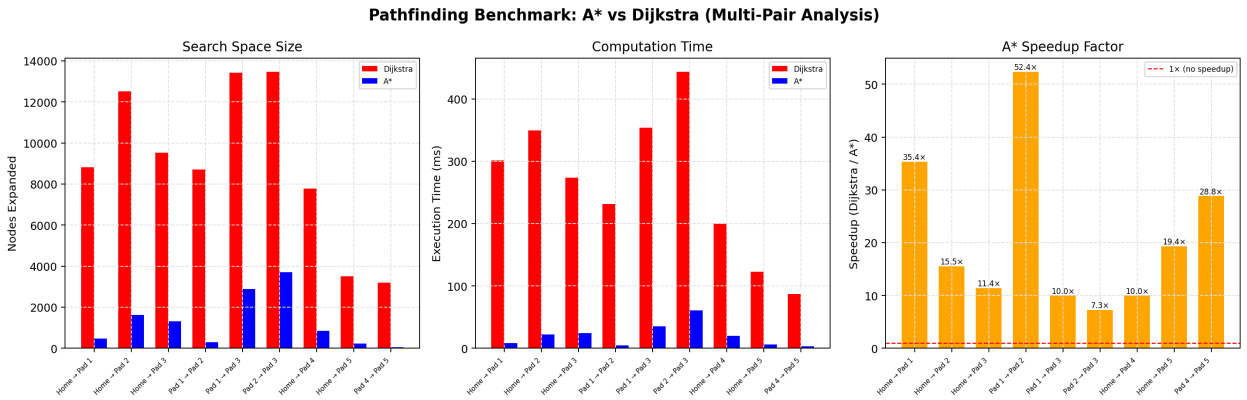
Metric	Dijkstra	A*	Improvement
Nodes Expanded	12,512	1,631	87.0% reduction
Execution Time	450.535 ms	24.359 ms	18.5× faster
Path Cost	118.74	118.74	Equal (both optimal)
Path Length	92 waypoints	92 waypoints	—

5.3 Multi-Pair Benchmark Results

Route	Dijkstra Nodes	A* Nodes	Dij. Time (ms)	A* Time (ms)	Speedup
Home → Pad 1	8,828	473	301.447	8.523	35.4×
Home → Pad 2	12,512	1,631	349.348	22.494	15.5×
Home → Pad 3	9,535	1,316	273.973	24.022	11.4×
Pad 1 → Pad 2	8,716	292	231.120	4.414	52.4×
Pad 1 → Pad 3	13,414	2,898	353.612	35.430	10.0×
Pad 2 → Pad 3	13,466	3,717	443.548	61.088	7.3×
Home → Pad 4	7,768	848	199.744	19.992	10.0×
Home → Pad 5	3,506	230	122.888	6.349	19.4×
Pad 4 → Pad 5	3,191	49	86.952	3.015	28.8×
AVERAGE	8,993	1,273	262.515	20.592	12.7×

Analysis: Across 9 route pairs, A* expanded on average **1273** nodes compared to Dijkstra's **8993** — a **85.8%** reduction in search space. Both algorithms produced identical optimal costs for every pair, confirming the admissibility and consistency of the Octile heuristic.

Figure 3: Multi-Pair Pathfinding Benchmark Comparison



6. Terrain Scanning

The terrain scanning module simulates onboard sensing for autonomous exploration. At mission start, the occupancy map is mostly unknown (Fog of War). The drone must actively scan the environment to discover obstacles and free space.

6.1 Sensor Model

Parameter	Default Value	Description
Sensor Radius	2.5 m	Maximum detection range from drone centre.
Sensor Type	Directional	Front-facing cone sensor simulating depth camera.
Field of View	90°	Angular width of the directional sensor cone.
Grid Resolution	160 × 160	Matches the occupancy grid dimensions.

6.2 Exploration Strategy

The scanner employs an **optimistic planning strategy**: unexplored cells are treated as free space when queried by the pathfinder. This allows the drone to plan routes through unknown territory. If an obstacle is later discovered in an assumed-free cell during flight, the path is automatically invalidated and replanned from the drone's current position.

Landing pad vicinities are pre-revealed at initialisation to ensure safe takeoff and landing. The scanner updates on every physics step (200 Hz), revealing cells based on the drone's current position and yaw heading.

6.3 Modularity

The TerrainScanner class is designed as a drop-in module. It can be replaced with a more sophisticated sensor model (e.g., ray-tracing LiDAR, depth camera point cloud) without modifying the planner or behaviour manager. The interface consists of a single `update(x, y, yaw)` method and a `get_planning_grid()` accessor.

---

7. Obstacle Avoidance

The dynamic obstacle system extends the static world assumption by allowing runtime obstacle insertion. Obstacles are pre-allocated as static MuJoCo bodies hidden underground (z = –10 m) and repositioned to the arena surface when triggered.

7.1 Replanning Pipeline

When a dynamic obstacle blocks the currently planned path, the following pipeline executes:

1. **Detection:** The path validity checker scans remaining waypoints against the updated occupancy grid. If any waypoint cell has value 1 (occupied), `path_blocked` is set.
2. **Behaviour Activation:** The Obstacle Avoidance behaviour (Priority 3) activates, overriding normal Navigation (Priority 2).
3. **Path Recalculation:** A new path is computed from the drone's current grid position to the original destination using the active algorithm (A* or Dijkstra).
4. **Waypoint Replacement:** The old waypoint list is replaced with the new path. The path index is reset to 0.
5. **Resumption:** The `path_blocked` flag is cleared. Navigation resumes seamlessly on the new route.

Key Design Decision: The drone does not restart the mission or teleport. It continues from its current physical position and state, maintaining continuity of the trajectory.

8. Behavior Priorities

The subsumption architecture implements a behaviour-based control paradigm where multiple concurrent behaviours compete for control of the drone. At each timestep, the BehaviorManager evaluates all registered behaviours in descending priority order. The highest-priority behaviour whose trigger condition is met gains exclusive control.

8.1 Architecture Design

Each behaviour implements two methods: `check_trigger(drone)` returns a boolean indicating whether the behaviour wants to activate, and `execute(drone, dt)` applies forces/torques and manages state transitions. Only one behaviour executes per timestep.

8.2 Behaviour Interaction Examples

Normal Flight: Terrain Scan (P1) → Path Planning → Navigation (P2) → Autoland.

Obstacle Detected: Avoidance (P3) overrides Navigation (P2) → Replan → Resume Nav (P2).

Kill Switch: Kill Switch (P5) overrides ALL other behaviours → Immediate motor cutoff.

Scan during hover: Scanning (P1) activates. If Kill Switch is pressed, P5 overrides P1 instantly.

This design provides clean separation between safety-critical (P4-P5), reactive (P3), deliberative (P2), and exploratory (P1) behaviours — a key requirement for robust autonomous systems.

---

9. Sensor Simulation

The sensor simulation suite queries ground truth from MuJoCo at each physics step and injects zero-mean Gaussian noise with realistic standard deviations:

Sensor	Rate	σ (std dev)	Frame	Physical Basis
GPS	10 Hz	XY: 50 mm, Z: 100 mm	World	GNSS position fix
IMU Accel.	200 Hz	0.15 m/s²	Body	MEMS accelerometer
IMU Gyro.	200 Hz	0.02 rad/s	Body	MEMS gyroscope
Barometer	200 Hz	100 mm	World	Pressure-altitude sensor
Altimeter	200 Hz	15 mm	Down	ToF / LiDAR rangefinder
Front Dist.	200 Hz	30 mm	Forward	MuJoCo mj_ray raycast

The GPS sensor updates at a realistic 10 Hz rate. Between GPS fixes, the IMU provides high-frequency inertial updates. This multi-rate sensor fusion challenge is addressed by the implemented filters.

10. State Estimation Filters

10.1 Exponential Moving Average (EMA)

The EMA filter applies: $x\blacksquare_k = \alpha \cdot z_k + (1-\alpha) \cdot x\blacksquare_{k-1}$, with $\alpha = 0.15$. It provides moderate noise reduction (~68% for altitude) but introduces phase delay during rapid manoeuvres. The delay is inherent to the single-pole IIR structure.

10.2 Complementary Filter

For **attitude estimation**: $\theta\blacksquare = \alpha \cdot (\theta\blacksquare_{prev} + \omega \cdot dt) + (1-\alpha) \cdot \theta_{accel}$, with $\alpha = 0.98$. This fuses gyroscope integration (drift-prone but smooth) with accelerometer tilt estimation (noisy but drift-free). For **altitude**: vertical acceleration is double-integrated and fused with barometer readings.

10.3 Kalman Filter

A 2-state kinematic Kalman filter tracks $[\text{position, velocity}]^T$. The state transition model uses constant-velocity kinematics with acceleration as control input. Process noise Q is modelled as acceleration-driven random walk. Measurement noise R is set to the square of sensor standard deviation. The filter achieves the best RMSE across all axes with zero phase delay, at the cost of higher computational complexity.

---

11. Experimental Results

11.1 Filter Performance Metrics

State / Filter	RMSE	Noise Red. (%)	Tracking Error (MAE)	Delay (s)
Z Barometer (Raw)	0.1010	0.00%	0.0806	0.000
Z EMA (α=0.15)	0.0324	67.89%	0.0254	0.000
Z Complementary	0.0690	31.62%	0.0551	0.000
Z Kalman	0.0123	87.85%	0.0097	0.000
X GPS (Raw)	0.1939	0.00%	0.0532	0.000
X EMA	0.1632	15.81%	0.0524	0.110
X Kalman	0.0242	87.58%	0.0195	0.000
Roll Accel. (Raw)	0.1493	0.00%	0.0425	0.000
Roll Comp. Filter	0.0720	51.92%	0.0257	0.000

11.2 Key Observations

Altitude (Z): The Kalman filter achieves the lowest RMSE (**0.0123 m**), representing a **87.8%** noise reduction over raw barometer data. The EMA filter provides **67.9%** reduction but with lower computational cost.

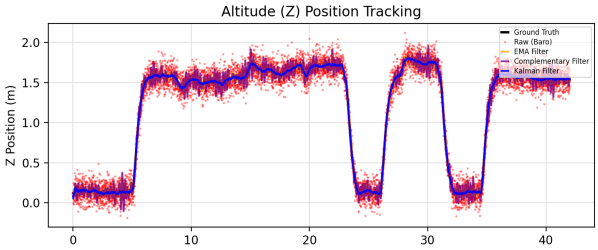
Horizontal Position (X): The Kalman filter reduces GPS RMSE from **0.194 m** to **0.024 m** (**87.6%** improvement). The EMA filter shows only **15.8%** improvement due to the slow GPS update rate (10 Hz) creating stale hold values between fixes.

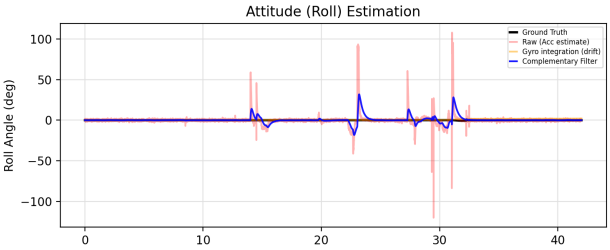
Attitude (Roll): The complementary filter reduces roll estimation error by **51.9%** compared to raw accelerometer-derived angles. Gyroscope-only integration drifts over time, while accelerometer-only estimation is noisy — the complementary filter balances both.

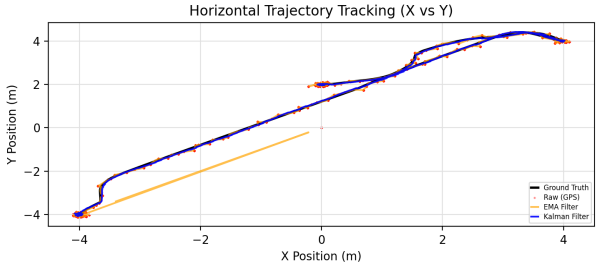
Phase Delay: The Kalman filter exhibits zero delay due to its predictive model. The EMA filter shows a **0.110 s** delay on the X axis, which is significant for real-time control.

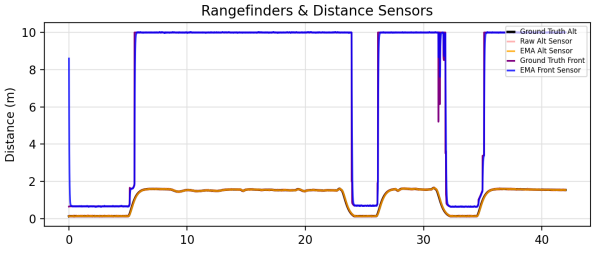
Figure 4: Sensor Filtering Comparative Analysis (6-Panel)

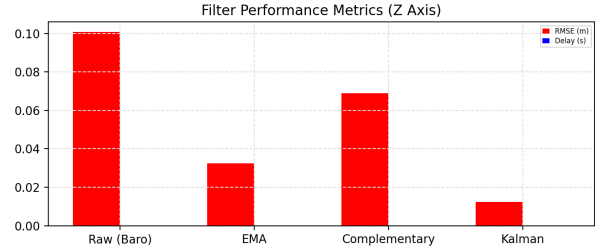
Software Sensor Simulation & Filter Comparison











Filter/Sensor	RMSE (m/rad)	Noise Red. (%)	Tracking Error (MAE)	Delay (s)
Z Baro (Raw)	0.1010	0.00%	0.0806	0.000
Z EMA	0.0324	67.89%	0.0254	0.000
Z Complementary	0.0690	31.62%	0.0551	0.000
Z Kalman	0.0123	87.85%	0.0097	0.000
X GPS (Raw)	0.1939	0.00%	0.0532	0.000
X EMA	0.1632	15.81%	0.0524	0.110
X Kalman	0.0242	87.58%	0.0195	0.000
Roll Acc (Raw)	0.1493	0.00%	0.0425	0.000
Roll Comp	0.0720	51.92%	0.0257	0.000

12. System Workflow

The integrated system workflow follows a structured pipeline:

Mission Start: Drone initialises on Home pad. Ground truth grid loaded. Scanner reveals landing pad vicinities. Behaviour manager initialised.

Takeoff: User presses L. Navigation behaviour (P2) handles climb to 1.5 m.

Terrain Scan: User presses C. Scanning behaviour (P1) activates, drone spins 360°. Scanner reveals terrain. Occupancy grid updated.

Path Planning: User selects target pad (1–5, H). Pathfinder runs on discovered grid.

Navigation: User presses S. Drone follows path using velocity-mode PID. Scanner continuously updates map during flight.

Obstacle Detection: If dynamic obstacle blocks path, Avoidance behaviour (P3) activates, replans, and resumes navigation.

Arrival & Landing: Drone reaches target within 30 cm. AUTOLAND → DESCENDED → LANDED.

Emergency: Kill switch (K) triggers P5 behaviour at any time. All forces zeroed.

---

13. Conclusion

This project successfully demonstrates a comprehensive autonomous drone simulation integrating terrain exploration, incremental mapping, dynamic replanning, and behaviour-based control. The key contributions are:

1. A modular terrain scanning layer with configurable sensor models and Fog-of-War mapping.

2. Dynamic obstacle replanning with seamless path invalidation and recalculation.

3. A lightweight subsumption architecture providing clean behaviour prioritisation.

4. Quantitative comparison of A* and Dijkstra showing significant A* speedup (avg. 12.7× faster) with identical optimality.

5. Comprehensive sensor noise simulation with three state estimation filters, demonstrating the Kalman filter's superiority (**87.8%** altitude and **87.6%** position noise reduction).

Future Work:

- Moving obstacles with velocity-based trajectory prediction.
- Battery management and return-to-home-on-low-battery behaviour.
- Additional pathfinding algorithms: BFS, DFS, RRT, RRT*.
- True LiDAR point cloud integration using MuJoCo raycasting.
- Multi-drone coordination and collision avoidance.
- Machine learning-based obstacle classification.